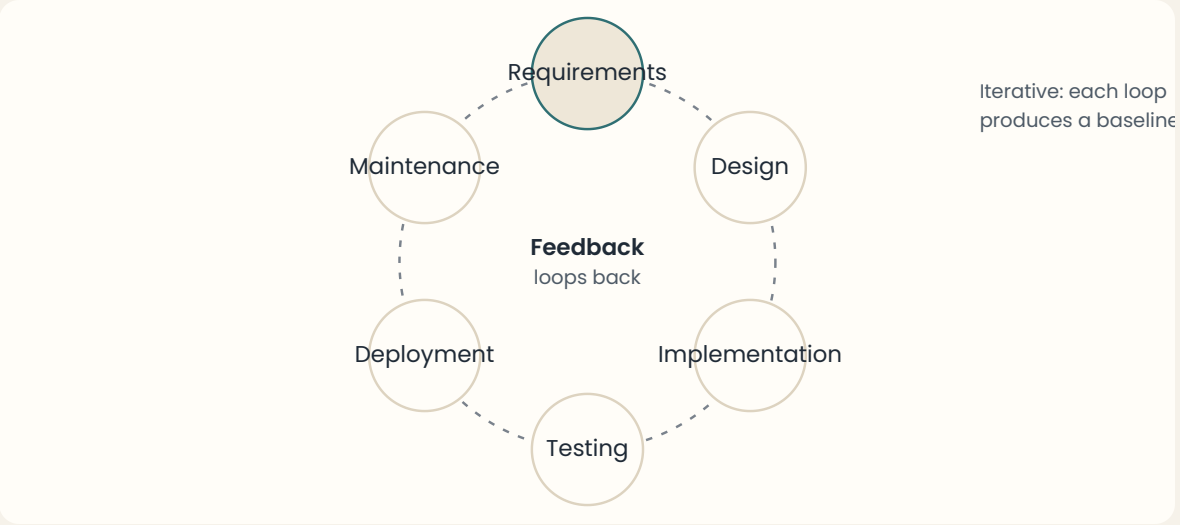
2

Software Development 1 · Week 2

Requirements engineering

A requirement says what the system must do or how well, not how it is built.



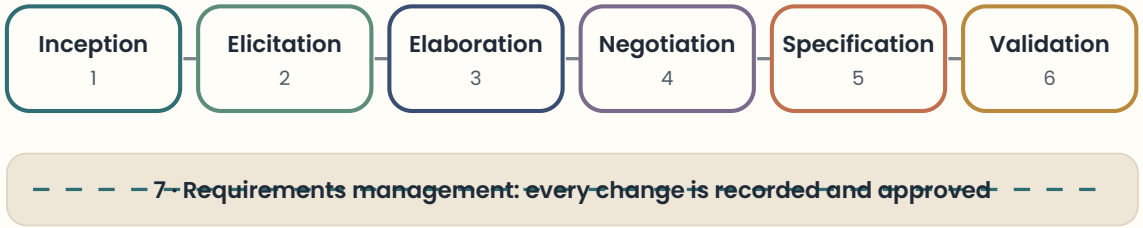
Think of it like this. A doctor does not prescribe medicine for "I feel bad". She asks questions until she finds the cause, because headache from stress and headache from an infection need different treatments. Requirements engineering is the diagnosis; code is the treatment.

Functional = what the system does. Non-functional = how well it does it, or under which constraints.

- Non-functional families: product (efficiency, usability, reliability, portability, security), organisational, external.
- FURPS+: Functionality, Usability, Reliability, Performance, Supportability, plus constraints.
- Every non-functional requirement needs a number you can test.
- User requirements speak to clients; system requirements give developers the detail.

> **Think of it like this.** Ordering a car: "it has five seats, a trunk, and air conditioning" are functional requirements (what it does). "It does 15 km per litre, reaches 100 km/h in 10 seconds, and passes the safety test" are non-functional (how well, and under which rules). Two cars with the same features can feel completely different because of the second list.

Core activities: feasibility, elicitation, analysis, specification, validation, management.



Think of it like this. Planning a wedding: first you check what the budget allows (feasibility), then you ask both families what they want (elicitation), resolve "outdoor" versus "grandma cannot walk on grass" (analysis and negotiation), write the plan (specification), show it to the couple (validation), and keep a log when the guest list changes for the fifth time (management).

Techniques: interviews, questionnaires, observation, workshops, document analysis, prototypes, scenarios.



Think of it like this. A journalist covering a story interviews witnesses (interviews), reads police reports (document analysis), visits the scene (observation), and then cross-checks the versions to remove contradictions before publishing (analysis). Publishing the first witness's version unchecked is how wrong stories, and wrong software, get made.

An SRS has an introduction, an overall description, specific requirements, and appendices.

- One idea per numbered requirement, consistent words, a glossary, a source, and a priority.
- Quality: correct, unambiguous, complete, consistent, verifiable, feasible, traceable, modifiable.
- Validate with reviews, prototypes, and early test cases, always with the client.

> **Think of it like this.** A recipe is a specification. "Add some salt and bake until done" is ambiguous and unverifiable; "add 5 g of salt and bake at 180 °C for 25 minutes, until a toothpick comes out clean" can be followed by anyone and checked at the end. Validation is the friend who tastes it before the party.

Enduring requirements are stable; volatile ones change (changing, emergent, consequential, compatibility).

- Every change: record, analyse impact, approve or reject, apply, new version.
- Traceability: source (where it came from), requirements (what depends on it), design (what implements and tests it).

> **Think of it like this.** Moving a load-bearing wall in a house: before you knock it down, you check the plans to see which pipes, cables, and rooms depend on it, you get the owner's signature, and you update the plans afterwards. Traceability is the plan that shows what depends on what; change management is the signature and the update.

Template: As a <role>, I want <goal>, so that <benefit> (who, what, why).

- Three Cs: Card, Conversation, Confirmation.
- INVEST: Independent, Negotiable, Valuable, Estimable, Small, Testable.
- Theme > epic > story > task. Tasks say how; stories say what and why.
- The backlog is ordered by value; MoSCoW sorts a release into Must, Should, Could, Won't.

> **Think of it like this.** A user story is like a sticky note on the fridge that says "Mum, buy milk for Sunday pancakes". It does not list brands or fat percentages; it reminds you to talk ("whole or skim?") and it says why, so if the shop has no milk you might buy pancake mix instead.

Acceptance criteria say when a story is done; agree on them before work starts.

- Given (context), When (event), Then (expected result); And adds more.
- Cover the happy path and the important unhappy paths with real example values.
- More than about five criteria? Split the story: by steps, rules, paths, roles, data, operations.

> **Think of it like this.** A driving test has clear acceptance criteria: given a parking space between two cars, when you park, then you finish in under three moves without touching either car. The examiner does not judge "drives well" by feeling; the criteria decide, and everyone knows them before the test starts.

Step by step

Put the process in order

1. Feasibility: check that budget and technology allow it
2. Elicitation: interview groomers and watch a busy Saturday
3. Analysis: resolve conflicts and agree priorities with the owner
4. Specification: write the numbered requirements and stories
5. Validation: review them with the owner using a prototype
6. Management: record and approve each later change

Check yourself

1. A client writes: "Store each order in PostgreSQL with a UUID key." What is it?

A) A design decision, not a requirement B) A functional requirement C) A business requirement

2. Which statement is a system requirement rather than a user requirement?

A) A returned copy that someone reserved waits 3 days on the hold shelf, then passes to the next reader in the queue. B) Readers can reserve books online. C) The library wants shorter waits for popular books.

3. A gym app goes live. A week later, the city's consumer-protection office says members must be able to cancel online as easily as they signed up. Which step was weak?

A) Stakeholder identification and elicitation: a regulator was missed B) Coding: the developers typed too slowly C) Nothing: laws are not requirements

Answers are in the speaker notes.

Practise next

Labs

- From complaint to requirements: a public library
- Requirements review: fix a weak specification
- A backlog for a gym app: stories, INVEST, and Given–When–Then

On the site